

Outbound policy gate for LiveKit SIP

A working prototype of the compliance layer that every outbound voice-AI customer builds themselves, and that no platform currently provides.

Three-minute demo · **Written point of view (PDF)**

The problem

Putting an AI agent on an outbound call is a solved media problem and an unsolved compliance problem.

Before a single digit is dialled, someone has to answer: is this number suppressed, is it inside the legal calling window *at the dialled number's location*, how many times have we already tried this week, and did the last call end in a way that starts a cooldown. During the call, someone has to notice a stop-contact request and act on it before the call ends, not after.

None of that exists as a platform primitive. Every team building outbound agents reimplements it in application code, where it can't be audited and where each implementation is subtly wrong in its own way.

This is what that layer looks like when you build it properly, and what building it revealed about where it belongs.

What it does

A pre-dial gate evaluates five rules and returns a structured decision before any LiveKit client is constructed:

Rule	Enforces	Authority
<code>phone_number</code>	Dialable, location-derivable number	—
<code>suppression</code>	Stop-contact / attorney-representation flags	TCPA 47 CFR 64.1200(d); FDCPA 15 U.S.C. 1692c(c); Reg F 12 CFR 1006.6(c)
<code>conversation_cooldown</code>	Cooldown after a confirmed live conversation	Reg F 12 CFR 1006.14(b)(2)(i)
<code>seven_in_seven</code>	7 attempts per rolling 7 days, aggregated across all of a contact's numbers	Reg F 12 CFR 1006.14(b)(2)(i)
<code>time_window</code>	8am–9pm local to the dialled number	TCPA 47 CFR 64.1200(c)(1); Reg F 12 CFR 1006.6(b)(1)(i)

Every decision — allow or deny — writes an audit row carrying the reason code, the rule, the inputs it was evaluated against, and the regulation it rests on.

During the call, a monitor watches final transcripts for cessation intent. On detection it writes a durable suppression, has the agent acknowledge, and terminates — in that order, because the suppression has to survive teardown. The next dial to that number is refused, citing what was actually said.

Measured on live calls: 39ms and 42ms from final transcript to durable suppression.

Design decisions worth knowing

Fail closed, everywhere. A conservative phrase list catches unambiguous cessation without a model call. Everything else goes to a classifier with a 2.5s budget against a measured p95 of 1.6s. Timeout, error, unparseable output, and ambiguous verdicts all resolve to suppression. A wrongly-suppressed contact costs one conversation; a missed cessation is a statutory violation.

No bypass, anywhere. There is no flag that skips, relaxes, or overrides a rule. `--now` injects an evaluation instant so time-dependent rules can be tested outside the calling window; records produced that way are permanently marked simulated, and the CLI's option surface is pinned by a test so a bypass flag can't be added unnoticed.

No rule reads a clock. Every rule takes `now` as a required keyword and rejects naive datetimes. There is exactly one wall-clock read on the pre-dial path, in the caller-side adapter. Proven at

runtime by injecting a `datetime` whose `now` raises, and statically by scanning package source.

Calling windows resolve against the dialled number, not the server. Tested across DST boundaries, the exact transition second, split area codes, non-observing zones, and five different server timezones producing identical verdicts.

Known limitation, and why it's the interesting one

The calling window derives its timezone from the area code, because that is the only signal available in application code. Number portability breaks this: a 212 mobile can live in California permanently.

This is not a fixable bug. The carrier knows the real routing and the line type; the application never can. Every customer building this layer in their own code is building the same defensible-but-incorrect thing — which is the clearest argument that it belongs in the telephony platform rather than above it.

The written point of view develops that argument.

Quickstart

```
❏  
bash  
uv sync  
cp .env.example .env      # fill in LiveKit, Twilio, and model provider keys  
uv run scripts/create_trunk.py  
uv run scripts/seed_gate.py
```

Then, in one terminal:

```
❏  
bash  
uv run agent.py dev
```

And in another:

```
❏  
bash  
uv run scripts/dial.py +13035550130      # DENY: suppressed  
uv run scripts/dial.py +12135550120      # ALLOW: clean history  
uv run scripts/dial.py +12135550120 --now 2026-07-15T07:30:00-07:00 # DENY: outside window
```

Exit codes: `0` allowed and dialled, `2` usage error, `3` denied by the gate.

Seed contacts are synthetic and use reserved `555-01xx` numbers.

Tests

```
□  
bash  
uv run pytest
```

180 tests. The ones worth reading are `tests/gate/test_dialed_number_timezone.py` for the timezone edge cases, `tests/test_clock_override.py` for the no-bypass guarantees, and `tests/test_cessation_monitor.py` for the write-before-hangup ordering.

Scope

This is a personal weekend project built to evaluate LiveKit SIP and the Agents framework. It is not production software. State is SQLite, there are no migrations, and it has been exercised on a handful of live calls rather than at volume.

`docs/sdk-facts.md` records the SDK surfaces this depends on, with file and line citations, including several that are only discoverable by reading the installed source.